

CENTRO UNIVERSITÁRIO UNIFECAP

ENTREGA CONTÍNUA DE UMA PLATAFORMA DE PEDIDOS EM MICROSSERVIÇOS

Do Docker Compose ao Kubernetes com observabilidade e CI/CD

Relatório Técnico — Parte Teórica
Cloud DevOps Engineering

São Paulo
2026

1 ARQUITETURA DE MICROSERVIÇOS E O PAPEL DO DEVOPS EM AMBIENTES CLOUD-NATIVE

O cenário descrito no desafio — cada parte do sistema subindo “como dá” em máquinas diferentes, sem padronização nem visibilidade sobre produção — é típico de uma aplicação que cresceu rápido sem que a infraestrutura acompanhasse. Arquitetura de microsserviços é o modelo em que uma aplicação é decomposta em serviços pequenos e independentes, cada um responsável por uma capacidade de negócio específica e comunicando-se por rede — no caso do Pedidos Veloz, os serviços de Pedidos, Pagamentos e Estoque, orquestrados por um API Gateway. A vantagem não é só técnica, é organizacional: cada serviço pode ser desenvolvido, testado e escalado de forma independente, reduzindo o efeito cascata de uma falha localizada.

Esse ganho de independência tem um custo: a complexidade que antes ficava dentro de um único processo passa a existir na rede, entre processos. É aí que entra o DevOps como prática — não como cargo. DevOps cloud-native aproxima quem desenvolve de quem opera por meio de automação (ambiente local reproduzível, deploy automatizado, observabilidade compartilhada), reduzindo o tempo entre uma mudança de código e sua chegada segura à produção, sem depender do conhecimento tácito de uma única pessoa.

2 CONTEINERIZAÇÃO: DOCKER E KUBERNETES, E QUANDO USAR CADA UM

Contêineres empacotam um serviço junto de tudo que ele precisa para rodar, isolando-o do restante da máquina hospedeira sem o custo de uma máquina virtual completa. O Docker resolve a primeira metade do problema do Pedidos Veloz: cada serviço ganha uma imagem própria, versionada, que roda de forma idêntica no notebook de um desenvolvedor e no servidor de produção. O Docker Compose resolve o problema imediato do desafio — hoje cada time sobe partes do sistema de um jeito diferente — ao descrever, em um único arquivo, como serviços, redes e volumes se relacionam localmente, permitindo subir todo o ecossistema com um comando.

Docker, isoladamente, não resolve o que acontece quando esses contêineres precisam rodar em produção, em múltiplas máquinas, se recuperar de uma falha e escalar sob demanda sem derrubar o serviço. Esse é o papel do Kubernetes: um orquestrador que decide em qual nó cada contêiner roda, reinicia o que falha, distribui tráfego entre réplicas e mantém o estado desejado do sistema descrito em manifests declarativos. Em resumo: Docker resolve “como empacotar e rodar um serviço”; Kubernetes resolve “como manter vários serviços rodando de forma confiável em escala”. Para a Loja Veloz, isso significa

Compose no dia a dia de desenvolvimento e Kubernetes exclusivamente em produção, onde a alta disponibilidade durante a campanha realmente importa.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Orquestração de contêineres

A documentação oficial do Kubernetes organiza a orquestração em torno de poucos objetos centrais: o Pod (menor unidade executável), o Deployment (garante réplicas de um Pod e gerencia atualizações), o Service (expõe um conjunto de Pods sob um endereço estável, já que Pods são efêmeros) e o par ConfigMap/Secret (separa configuração e credenciais do código, alinhado ao princípio de configuração externa da metodologia 12-Factor App). Soma-se a isso o HorizontalPodAutoscaler (HPA), que observa uma métrica — normalmente uso de CPU — e ajusta o número de réplicas automaticamente, mecanismo essencial para absorver o pico de tráfego previsto na campanha da Loja Veloz sem intervenção manual.

3.2 CI/CD em ambientes distribuídos

Em um sistema com múltiplos serviços independentes, um pipeline único e manual não escala operacionalmente. Ferramentas como GitHub Actions permitem descrever, por serviço, um fluxo de build, teste e publicação de imagem disparado a cada alteração de código, com portas de qualidade (testes, lint, verificação de vulnerabilidade) antes de qualquer imagem chegar ao registry — reduzindo o principal risco citado no desafio, indisponibilidade durante deploys, já que a publicação deixa de depender de um passo manual sujeito a erro humano.

3.3 Observabilidade: métricas, logs e traces

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir do que ele expõe externamente, apoiada em três sinais complementares: métricas (séries numéricas, como uso de CPU ou taxa de erro), logs (registros discretos de eventos) e traces (o caminho que uma requisição percorre entre serviços, com a duração de cada etapa). O trace é o sinal que resolve diretamente a “baixa rastreabilidade de falhas entre serviços” apontada no desafio: sem ele, descobrir que uma lentidão no Estoque está atrasando a confirmação de um Pedido exige investigação manual, log por log. O padrão OpenTelemetry, mantido pela CNCF, unifica a coleta desses três sinais em formato vendor-neutro.

4 EXEMPLO CONCRETO DE REFERÊNCIA

Como referência de mercado, este trabalho usa o Online Boutique, aplicação de demonstração open source mantida pela Google Cloud Platform: um e-commerce composto por cerca de onze microsserviços — catálogo, carrinho, checkout, pagamento, frete —, escritos em linguagens diferentes e comunicando-se via gRPC, publicado com manifests Kubernetes prontos e pensado para demonstrar Kubernetes/GKE, service mesh e telemetria de ponta a ponta num cenário de compra online. O paralelo com o Pedidos Veloz é direto: assim como o Online Boutique separa catálogo, carrinho e pagamento em serviços distintos, este projeto separa Pedidos, Pagamentos e Estoque, adotando a mesma lógica de manifests versionados e observabilidade de ponta a ponta em vez de inspeção manual.

5 JUSTIFICATIVA DAS PRINCIPAIS DECISÕES ARQUITETURAIS

Três decisões guiaram o desenho técnico deste projeto. Primeiro, a comunicação entre Pedidos e os serviços de Pagamentos/Estoque é assíncrona, via evento “PedidoCriado” publicado em um barramento de mensagens, e não por chamada HTTP em cadeia — evitando que uma lentidão no gateway de pagamento externo derrube a criação de um pedido. Segundo, o deploy progressivo (canário) foi reservado para o serviço de Pedidos, por concentrar o maior risco de negócio, enquanto os demais usam rolling update padrão — decisão de custo-benefício para um MVP de quatro semanas. Terceiro, o HPA foi aplicado aos dois serviços com maior probabilidade de pico na campanha — Pedidos e Estoque —, deixando Gateway e Pagamentos com réplica fixa neste primeiro momento.

REFERÊNCIAS

GOOGLE CLOUD PLATFORM. microservices-demo (Online Boutique). Repositório GitHub, 2024. Disponível em: <https://github.com/GoogleCloudPlatform/microservices-demo>. Acesso em: 23 jul. 2026.

KUBERNETES. Documentação oficial. Disponível em: <https://kubernetes.io/docs/home/>. Acesso em: 23 jul. 2026.

DOCKER. Documentação oficial. Disponível em: <https://docs.docker.com/>. Acesso em: 23 jul. 2026.

THE TWELVE-FACTOR APP. Metodologia para aplicações cloud-native. Disponível em: https://12factor.net/pt_br/. Acesso em: 23 jul. 2026.

OPENTELEMETRY. Documentação oficial. Disponível em: <https://opentelemetry.io/docs/>. Acesso em: 23 jul. 2026.

GITHUB. GitHub Actions Documentation. Disponível em: <https://docs.github.com/actions>. Acesso em: 23 jul. 2026.